AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych

Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

Lista dwukierunkowa

Autor:
Arkadiusz Górski

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1 Ogólne określenie wymagań **2**
1.1 Opis i cel projektu . . . 2
1.2 Zasada działania listy dwukierunkowej . . . 2
1.3 Wykorzystanie systemu kontroli wersji Git oraz platformy GitHub . . . 3
1.3.1 Istota i cel stosowania systemu Git . . . 3
1.3.2 Adres zdalnego repozytorium projektu . . . 3
1.3.3 Wykorzystane komendy i cykl pracy (Workflow) . . . 4
1.4 Automatyczna generacja dokumentacji przy użyciu narzędzia Doxygen . . . 4
1.4.1 Istota działania oraz specyfika komentarzy dokumentacyjnych . . . 4
1.4.2 Zalety stosowania narzędzia Doxygen w projekcie . . . 6
1.4.3 Wady, ograniczenia i wyzwania związane z Doxygenem . . . 6
1.4.4 Podsumowanie wykorzystania Doxygena . . . 7

2 Analiza problemu **8**
2.1 Analiza operacji i wyzwań algorytmicznych . . . 8
2.2 Analiza zarządzania pamięcią na stercie . . . 8

3 Projektowanie **9**
3.1 Opis poszczególnych komponentów . . . 9
3.2 Powiązania i współpraca klas . . . 9

4 Implementacja **10**
4.1 Testowanie i weryfikacja w funkcji głównej . . . 10

5 Wnioski **11**
5.1 Podsumowanie realizacji projektu . . . 11
5.2 Aspekt dokumentacyjny i organizacyjny . . . 11
5.3 Praca z systemem kontroli wersji . . . 11
5.3.1 Deklaracja usprawnień w przyszłych projektach . . . 11

1 Ogólne określenie wymagań

1.1 Opis i cel projektu

Głównym celem projektu jest zaimplementowanie w języku C++ struktury danych w postaci dynamicznej listy dwukierunkowej alokowanej na sterce. Projekt ma na celu ugruntowanie wiedzy z zakresu zaawansowanego programowania obiektowego ze szczególnym uwzględnieniem stosowania wzorców projektowych:

- Wzorzec Fabryki - wykorzystywany do hermetyzacji procesu tworzenia nowych węzłów listy,
- Wzorzec Iteratora – umożliwiający bezpieczne, sekwencyjne przechodzenie po elementach struktury w obu kierunkach (do przodu i do tyłu)

Dodatkowo celami towarzyszącymi projektowi są:

- Praktyczne wykorzystanie systemu kontroli wersji Git oraz platformy GitHub w zarządzaniu historią zmian kodu
- Generowanie automatycznej dokumentacji kodu źródłowego przy użyciu narzędzia Doxygen.

1.2 Zasada działania listy dwukierunkowej

Zanim przejdziemy do szczegółowych wymagań i implementacji, warto przybliżyć samą naturę problemu oraz mechanizmy, na których opiera się stworzony program.

Tradycyjne tablice znane z języka C++ mają jedną poważną wadę ich rozmiar jest zazwyczaj sztywno ustalony podczas tworzenia, a wstawienie lub usunięcie elementu ze środka takiej tablice wymaga przesunięcia wszystkich pozostałych elementów w pamięci, co przy dużych zbiorach danych jest bardzo niewydajne. Rozwiązaniem tego problemu są struktury dynamiczne oparte na wskaźnikach, a jedną z najelastyczniejszych z nich jest właśnie lista dwukierunkowa.

W przeciwieństwie do tablicy, lista dwukierunkowa nie znajduje się w jednym, ciągłym bloku pamięci. Składa się z niezależnych od siebie obiektów zwanych węzłami, które są rozproszone w pamięci operacyjnej komputera (na tzw. stercie). Każdy taki węzeł przechowuje trzy kluczowe informacje:

- Wartość (dane), którą w naszym projekcie reprezentuje liczba całkowita.
- Wskaźnik o nazwie next, który wskazuje na kolejny węzeł w kolejności (umożliwiając poruszanie się do przodu).
- Wskaźnik o nazwie prev, który wskazuje na poprzedni węzeł (umożliwiając poruszanie się do tyłu).

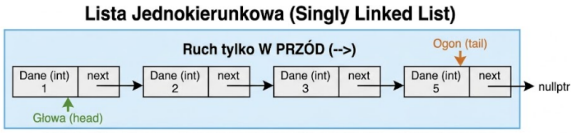
Całą listą zarządza główny obiekt, który pamięta tylko dwa skrajne punkty odniesienia:

- Wskaźnik head – wskazuje na pierwszy element listy.
- Wskaźnik tail – wskazuje na ostatni element listy.

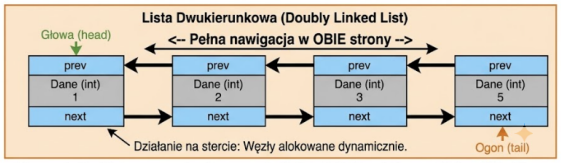
Dzięki takiej budowie, dodanie elementu na początek lub koniec listy nie wymaga przesuwania całej struktury wystarczy jedynie odpowiednio przepiąć wskaźniki sąsiadujących węzłów.

Przykładowo, kiedy dodajemy nowy element na początek, nowy węzeł staje się nową głową, jego wskaźnik wskazuje na dotychczasowy pierwszy element, a pole ”prev”
starego elementu zaczyna wskazywać na nasz nowy węzeł.

Warto również wspomnieć o zarządzaniu pamięcią na stercie. Ponieważ węzły są tworzone dynamicznie w trakcie działania programu (za pomocą operatora new, po zakończeniu pracy lub wyczyszczeniu listy (metoda clear) każdy węzeł musi zostać jawnie usunięty z pamięci za pomocą operatora delete. Zapobiega to groźnym wyciekom pamięci RAM.



Rysunek 1a: Prosty węzeł z jednym wskaźnikiem. Brak możliwości powrotu.



Rysunek 1b: Złożony węzeł z dwoma wskaźnikami. Pełna elastyczność i wydajność.

1.3 Wykorzystanie systemu kontroli wersji Git oraz platformy GitHub

W ramach realizacji projektu kluczowym aspektem inżynieryjnym było zastosowanie rozproszonego systemu kontroli wersji **Git** oraz platformy **GitHub** do zarządzania historią zmian, archiwizacji oraz publikacji kodu źródłowego.

1.3.1 Istota i cel stosowania systemu Git

Git jest rozproszonym systemem kontroli wersji, który umożliwia śledzenie zmian w plikach źródłowych na przestrzeni czasu. W przeciwieństwie do tradycyjnych metod archiwizacji (polegających na ręcznym tworzeniu kopii zapasowych całych katalogów, Git tworzy migawki całego projektu przy każdym zatwierdzeniu zmian (*commit*). Główne zalety i cele stosowania systemu Git w procesie wytwarzania oprogramowania to:

- **Bezpieczeństwo i cofanie zmian:** Możliwość powrotu do dowolnego wcześniejszego stanu aplikacji w przypadku popełnienia błędu lub awarii kodu (np. podczas implementacji złożonych wskaźników lub iteratorów).
- **Przejrzystość historii:** Dokładna dokumentacja tego, kto, kiedy i jakie modyfikacje wprowadził w kodzie źródłowym.
- **Współpraca i integracja:** Platformy takie jak GitHub umożliwiają zdalne przechowywanie repozytorium w chmurze, co ułatwia backup oraz dzielenie się efektem pracy z prowadzącym zajęcia.

1.3.2 Adres zdalnego repozytorium projektu

Kompletny kod źródłowy aplikacji, wraz z plikami nagłówkowymi, modułami fabryki, iteratora oraz pełną historią zatwierdzeń, został opublikowany w publicznym repozytorium na platformie GitHub pod adresem:

https://github.com/Ripurek/lista

1.3.3 Wykorzystane komendy i cykl pracy (Workflow)

Podczas realizacji projektu posłużono się podstawowym, liniowym cyklem pracy z systemem Git. W procesie tworzenia aplikacji wykorzystano następujące komendy konsolowe:

- `git status` – służy do bieżącej weryfikacji stanu repozytorium. Pozwala na zidentyfikowanie zmodyfikowanych, nowo utworzonych lub nieśledzonych przez system plików w katalogu roboczym.
- `git add .` – dodaje wszystkie pliki (lub wskazane modyfikacje) do tzw. strefy przejściowej (*staging area*), przygotowując je do trwałego zapisania w historii projektu.
- `git commit -m "opis"` – tworzy punkt kontrolny (commit) w lokalnej historii repozytorium. Do każdego zatwierdzenia dołączany jest zwięzły komunikat tekstowy opisujący charakter zmian (np. *"Naprawa logiki dekrementacji w iteratorze"* lub *"Dodanie komentarzy Doxygen"*).
- `git push origin main` – przesyła lokalnie zatwierdzone zmiany na zdalny serwer (platformę GitHub), zapewniając synchronizację oraz bezpieczeństwo danych w chmurze.
- `git log --oneline` – umożliwia podgląd uproszczonej historii wszystkich commitów, co pozwala na szybką weryfikację kolejnych etapów rozwoju projektu.

Praktyczne wykorzystanie systemu Git, mimo początkowych trudności związanych z nauką narzędzia, znacząco usprawniło proces testowania i rozwijania struktury listy dwukierunkowej, stanowiąc cenne doświadczenie inżynierskie.

1.4 Automatyczna generacja dokumentacji przy użyciu narzędzia Doxygen

Kolejnym kluczowym elementem inżynieryjnym w procesie realizacji projektu było zastosowanie narzędzia **Doxygen** wiodącego standardu w środowisku języka C++ do automatycznego generowania dokumentacji technicznej bezpośrednio z kodu źródłowego. Wraz ze wzrostem złożomości aplikacji, ręczne pisanie dokumentacji w osobnych edytorach tekstu staje się wysoce nieefektywne i podatne na błędy (np. rozbieżności między faktycznym działaniem kodu a jego opisem). Doxygen rozwiązuje ten problem, łącząc kod źródłowy z warstwą opisową w jeden spójny dokument.

1.4.1 Istota działania oraz specyfika komentarzy dokumentacyjnych

Doxygen nie jest klasycznym edytorem tekstu Analizuje on strukturę plików nagłówkowych oraz implementacyjnych (w tym projekcie m.in. `node.h`, `Iterator.h`, `DoublyLinkedList.h`) w poszukiwaniu specjalnie sformatowanych bloków komentarzy, zwanych komentarzami dokumentacyjnymi.

W przeciwieństwie do zwykłych komentarzy programistycznych (które służą do krótkiego wyjaśnienia logiki wewnątrz funkcji dla innych programistów), komentarze dla Doxygena muszą spełniać rygorystyczne reguły składniowe oraz używać odpowiednich znaczników semantycznych. W projekcie wykorzystano standardowe formaty:

- Potrójny ukośnik (`///`) lub linijkowy komentator z wykrzyknikiem (`//!`) dla krótkich, jednolinijkowych opisów pól klasy lub metod,
- Blokowe komentarze z podwójną gwiazdką (`/** ... */` lub `/*! ... */`) dla rozbudowanych nagłówków klas i złożonych procedur algorytmicznych.

Wewnątrz takich bloków stosuje się tzw. znaczniki strukturalne (ang. *tags*), które pozwalają precyzyjnie opisać poszczególne parametry kodu. W niniejszej dokumentacji kodu zastosowano m.in.

- `brief` – krótki, jednozdaniowy opis podsumowujący rolę danej klasy lub metody,
- `details` – szczegółowy opis rozwijający logikę działania algorytmu (np. mechanizmu iteracji w obie strony lub alokacji w fabryce),
- `param nazwa` – precyzyjny opis konkretnego argumentu przekazywanego do metody,
- `return` – informacja o typie i znaczeniu wartości zwracanej przez funkcję,
- `tparam` – opis parametrów szablonu (w strukturach generycznych).

1.4.2 Zalety stosowania narzędzia Doxygen w projekcie

Wprowadzenie Doxygena do cyklu wytwórczego listy dwukierunkowej przyniosło szereg istotnych korzyści o charakterze inżynieryjnym:

- **Automatyzacja i oszczędność czasu:** Generowanie dokumentacji (uwzględniającej wykresy relacji klas, hierarchię dziedziczenia oraz spisy metod) odbywa się automatycznie poprzez odpytanie parsera, co eliminuje potrzebę ręcznego tworzenia diagramów od podstaw.
- **Synchronizacja kodu z dokumentacją:** Ponieważ komentarze znajdują się bezpośrednio w plikach nagłówkowych tuż obok logiki algorytmicznej, znacznie łatwiej utrzymać ich aktualność. Każda zmiana w kodzie naturalnie prowokuje programistę do weryfikacji opisu.
- **Wieloplatformowość i elastyczność formatów wyjściowych:** Narzędzie potrafi wygenerować czytelną dokumentację w formacie HTML (z interaktywną wyszukiwarką i drzewem plików) oraz w formacie LaTeX, który po kompilacji tworzy profesjonalny raport PDF.
- **Wizualizacja powiązań architektonicznych:** Dzięki integracji z pakietem Graphviz, Doxygen automatycznie generuje diagramy współpracy klas co drastycznie podnosi walory wizualne i analityczne dokumentacji projektowej.

1.4.3 Wady, ograniczenia i wyzwania związane z Doxygenem

Mimo bezdyskusyjnych zalet, praca z Doxygenem wiąże się z konkretnymi wyzwaniami, trudnościami i ograniczeniami technznymi:

- **Rygorystyczna składnia i specyficzne komentarze:** Doxygen jest niezwykle wrażliwy na poprawność składniową znaczników. Przypadkowa literówka w komendzie (np. wpisanie
`prm` zamiast
`param`) powoduje, że dokumentacja nie interpretuje poprawnie argumentu, a ostrzeżenia parsera bywają lakoniczne. Wymaga to dużej dyscypliny pisarskiej.
- **Wysoki próg wejścia i złożoność konfiguracji:** Plik konfiguracyjny `Doxyfile` generowany poleceniem `doxygen -g` zawiera ponad kilkaset linii parametrów, ścieżek, przełączników kodowania oraz opcji językowych. Poprawne skonfigurowanie go tak, aby bezbłędnie obsługiwał polskie znaki kodowania UTF-8 i generował estetyczny kod LaTeX, wymagało czasochłonnych prób i testów.
- **Problem „martwej" dokumentacji (Stale Documentation):** Jeśli programista zmodyfikuje algorytm wewnątrz metody (np. zmieni warunki brzegowe w iteratorze), ale zapomni zaktualizować blok komentarza Doxygena, wygenerowany raport staje się dezinformujący, wprowadzając błąd poznawczy u osoby analizującej kod.
- **Narzucony styl i ograniczona elastyczność szablonów:** Domyślne szablony graficzne generowane przez Doxygena bywają surowe, a ich zaawansowana modyfikacja (zwłaszcza w wersji HTML) wymaga dodatkowej znajomości arkuszy stylów CSS oraz struktury konfiguracji programu.

1.4.4 Podsumowanie wykorzystania Doxygena

Podsumowując, implementacja oraz konfiguracja narzędzia Doxygen w projekcie listy dwukierunkowej stanowiła cenne doświadczenie inżynierskie. Mimo początkowych trudności z opanowaniem specyficznej składni znaczników oraz konfiguracją pliku `Doxyfile`, automatyzacja procesu tworzenia dokumentacji technicznej pozwoliła na uzyskanie pełnego, spójnego i profesjonalnego zestawienia struktur klasowych, co bezpośrednio podniosło wartość merytoryczną całej pracy.



2 Analiza problemu

Identyfikacja problemu i założeń projektowych Głównym problemem badawczym i implementacyjnym w tym projekcie jest zaprojektowanie oraz zaimplementowanie efektywnej struktury danych w postaci listy dwukierunkowej w języku C++. W przeciwieństwie do standardowych tablic o stałym rozmiarze, struktura ta musi radzić sobie z dynamicznym narastaniem i kurczeniem się zbioru danych w pamięci operacyjnej , bez konieczności alokowania jednego, ciągłego bloku pamięci.

Kluczowym wyzwaniem jest takie powiązanie pojedynczych elementów, aby operacje dodawania, usuwania oraz przeglądania danych odbywały się sprawnie, a narzut związany z zarządzaniem wskaźnikami był jak najmniejszy.

2.1 Analiza operacji i wyzwań algorytmicznych

Podczas analizy zachowania listy dwukierunkowej zidentyfikowano kilka kluczowych scenariuszy wymagających szczególnej uwagi.

- Obsługa przypadków brzegowych

  Dodawanie lub usuwanie elementów w sytuacji, gdy lista jest całkowicie pusta, gdy zawiera tylko jeden element, albo gdy operacja dotyczy samego początku lub końca struktury. Wymaga to precyzyjnej aktualizacji wskaźników głównego zarządcy

- Nawigacja dwukierunkowa

  Zapewnienie możliwości płynnego przechodzenia po elementach zarówno do przodu, jak i do tyłu. Wymagało to przeanalizowania struktury pod kątem przechowywania dwóch wskaźników oraz zaprojektowania mechanizmu iteratora obsługującego operatory inkrementacji i dekrementacji

- Hermetyzacja tworzenia obiektów: Analiza wykazała potrzebę oddzielenia logiki samej listy od procesu powoływania nowych węzłów do życia, co uzasadniło użycie wzorca fabryki.

2.2 Analiza zarządzania pamięcią na stercie

Ponieważ wszystkie węzły listy są alokowane dynamicznie w pamięci RAM za pomocą operatora „new”, istotnym problemem staje się ryzyko wystąpienia wycieków pamięci. Analiza wykazała konieczność zaprojektowania mechanizmów sprzątających

- Każdy dynamicznie utworzony węzeł musi zostać w odpowiednim momencie zwolniony przy użyciu operatora delete

- Wymagane jest zaimplementowanie metody czyszczącej (clear), która bezpiecznie usuwa wszystkie elementy po kolei od początku do końca, zapobiegając pozostawianiu ”osieroconych”wskaźników.

3 Projektowanie

Projekt został podzielony na niezależne moduły zgodnie z wymogami modularności kodu. Każda klasa znajduje się w osobnym pliku nagłówkowym oraz pliku źródłowym co zapewnia pełną czytelność projektu oraz ułatwia jego rozwój i testowanie. Architektura opiera się na czterech głównych komponentach współpracujących ze sobą w celu zapewnienia pełnej funkcjonalności listy dwukierunkowej.

3.1 Opis poszczególnych komponentów

- Klasa Node odpowiada za pojedynczy element przechowywany w strukturze. Obiekt ten przechowuje wartość typu całkowitego oraz dwa wskaźniki skierowane na sąsiednie elementy umożliwiające ruch w obie strony.
- Klasa NodeFactory realizuje wzorzec fabryki odpowiedzialny za proces tworzenia nowych obiektów. Jej zadaniem jest dynamiczne powoływanie do życia nowych węzłów na stercie oraz ich wstępna inicjalizacja dzięki czemu logika alokacji pamięci zostaje całkowicie odseparowana od zarządzania samą listą.
- Klasa ListIterator implementuje wzorzec iteratora. Działa ona jako inteligentny wskaźnik który enkapsuluje aktualną pozycję w strukturze i udostępnia przeciążone operatory inkrementacji oraz dekrementacji pozwalające na płynne poruszanie się po elementach w przód i w tył.
- Klasa DoublyLinkedList stanowi główny zarządca całej struktury. Przechowuje ona wskaźniki wyznaczające początek oraz koniec listy i udostępnia kompletny interfejs metod operujących na danych takich jak dodawanie usuwanie przeglądanie oraz czyszczenie pamięci.

3.2 Powiązania i współpraca klas

Zarządzanie strukturą opiera się na ścisłej współpracy pomiędzy zdefiniowanymi komponentami. Klasa DoublyLinkedList zleca klasie NodeFactory utworzenie nowego węzła Node zawsze wtedy gdy użytkownik dodaje element do listy. Z kolei do bezpiecznego przeglądania zawartości struktury powoływany jest ListIterator który operuje bezpośrednio na wskaźnikach powiązanych węzłów bez ujawniania szczegółów implementacyjnych na zewnątrz.

4 Implementacja

Zgodnie z wymaganiami projektowymi kod źródłowy został podzielony na osobne moduły. Każda klasa posiada dedykowany plik nagłówkowy zawierający deklaracje oraz plik źródłowy implementujący logikę metod. W projekcie znajdują się następujące pliki

- node.h oraz node.cpp odpowiedzialne za definicję pojedynczego węzła listy,
- NodeFactory.h oraz NodeFactory.cpp realizujące wzorzec fabryki obiektów,
- Iterator.h oraz Iterator.cpp implementujące dwukierunkowy iterator,
- DoublyLinkedList.h oraz DoublyLinkedList.cpp zawierające główną logikę zarządzania strukturą,
- main.cpp grupujący procedury testowe weryfikujące poprawność działania wszystkich metod.

4.1 Testowanie i weryfikacja w funkcji głównej

Wszystkie zaimplementowane funkcjonalności zostały poddane testom. Przeprowadzone scenariusze objęły dodawanie elementów na oba końce oraz pod wskazany indeks, usuwanie danych, wyświetlanie zawartości w obu kierunkach przy użyciu iteratora oraz końcowe czyszczenie pamięci. Testy potwierdziły poprawność logiczną oraz stabilność działania aplikacji

5 Wnioski

5.1 Podsumowanie realizacji projektu

Zrealizowany projekt pozwolił na praktyczne opanowanie zaawansowanych mechanizmów programowania obiektowego w języku C++. Sam zamysł i architektura programu opartego na liście dwukierunkowej oraz wzorcach projektowych nie są skomplikowane w założeniach, jednak proces ich pełnego zrozumienia i poprawnego zaimplementowania okazał się niezwykle czasochłonny. Wymagało to skrupulatnego kontrolowania wskaźników oraz zarządzania pamięcią sterty, co bezpośrednio przełożyło się na solidne ugruntowanie wiedzy teoretycznej.

5.2 Aspekt dokumentacyjny i organizacyjny

Dużą częścią całego przedsięwzięcia było przygotowanie formalnej dokumentacji technicznej i projektowej. Tworzenie opisu wymagań, analizy problemu, projektowania oraz generowanie dokumentacji za pomocą narzędzia Doxygen pochłonęło znacznie więcej czasu, niż początkowo zakładano. Pokazuje to, że rzetelne udokumentowanie kodu jest równie wymagającym zadaniem, co sama implementacja algorytmów.

5.3 Praca z systemem kontroli wersji

Istotnym doświadczeniem była również praca z systemem kontroli wersji Git oraz platformą GitHub. Ponieważ była to pierwsza styczność z tym narzędziem, całościowy proces obsługi repozytorium, wykonywania commitów, cofania zmian czy obsługi gałęzi stanowił duże wyzwanie. Mimo pomyślnego zrealizowania wszystkich wymaganych scenariuszy testowych, ze względu na brak wcześniejszego doświadczenia wciąż nie czuję się w pełni swobodnie i pewnie w codziennym użytkowaniu Gita, co wskazuje na obszar wymagający dalszej praktyki i nauki.

5.3.1 Deklaracja usprawnień w przyszłych projektach

Na podstawie wyciągniętych wniosków oraz doświadczeń zebranych w trakcie realizacji niniejszego projektu, w kolejnych przedsięwzięciach programistycznych planuję wdrożyć następujące dobre praktyki:

- **Praca w cyklu iteracyjnym:** Równoległe tworzenie kodu, testów jednostkowych oraz dokumentacji w krótkich, 1/2 dniowych odstępach czasowych, zamiast odkładania pisania raportów na sam koniec prac.
- **Wcześniejsza konfiguracja narzędzi:** Przygotowanie pliku `Doxyfile` oraz środowiska Git już na etapie inicjalizacji repozytorium, co pozwoli na płynne dokumentowanie kodu od pierwszego commita.
- **Skupienie na detalach i jakości opisu:** Przeznaczenie stałego czasu na Dopieszczanie warstwy wizualnej i językowej dokumentacji, aby tworzone opracowania były nie tylko merytoryczne, ale również czytelne i estetyczne.

Słowniczek pojęć

Parser Zaawansowany program lub komponent, który analizuje strukturę logiczną kodu źródłowego.